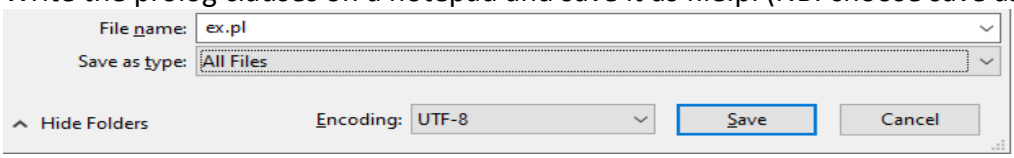
23CSE401 Fundamentals of AI – Lab Exercises in Prolog

- Install swi-prolog
- Write the prolog clauses on a notepad and save it as file.pl (NB: choose save as type:



- Open swi-prolog -> file -> consult (choose the file.pl).
- Enter your queries
- ?-write('\33\[2J'). % To clear a screen

Ex1
Consult likes.pl
?- consult(['d:/likes.pl']).

```
likes(john, pizza).
likes(john, pasta).
likes(mary, sushi).
likes(mary, pizza).
likes(X, Y) :- likes(X, Z), likes(Z, Y).    % A simple rule
grandfather(X, Y) :- father(X, Z), parent(Z, Y).    % A simple rule

?-listing(likes) % to view contents
?-write('\33\[2J'). % To clear a screen
?- likes(mary, X). % to query
```

For next enter **semicolon**
To abort : ctrl C and then enter “a”

```
?-likes(Y, pizza).
 ?- family_tree1:listing.
```

Ex2 family_tree

?- consult(['d:/family_tree.pl']).

```
% Gender facts
male(jack).
male(oliver).
male(james).
female(helen).
female(sophie).
female(jess).

%  Parent relationships (parent(Parent, Child))
parent(jack, jess).
parent(helen, jess).
parent(oliver, james).
parent(sophie, james).
```

2. The Rules (The Logic)

Rules use variables (words starting with a capital letter) to define relationships. The :- symbol means "if", and the comma , means "and".

```
% A mother is a parent who is female
mother(X, Y) :-  parent(X, Y), female(X).
% A father is a parent who is male
father(X, Y) :- parent(X, Y), male(X).
% Two people are siblings if they share the same parents and are not the same person
sibling(X, Y) :- parent(Parent, X), parent(Parent, Y),  X \== Y.
```

3. Running Queries

?- family_tree1:listing.
After loading the facts and rules into a Prolog interpreter, you can ask the system questions about your family tree.

- Query 1: Who is the father of james?
  - Code: ?- parent(X, james).
  - Output: X = oliver
  - *Semicolon for next parent*
- Query 2: Who is the sibling of jess?
  - Code: ?- sibling(X, jess).
  - Output: X = lily

Ex3. Family Tree Queries with Variable Bindings

```
parent(bob, lisa).
parent(bob, tim).
parent(lisa, mary).
parent(tim, john).

% Rules
sibling(X, Y) :- parent(P, X), parent(P, Y), X \= Y.
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
Query: Who are Bob's grandchildren?
?- grandparent(bob, Grandchild).
% Output:
% Grandchild = mary ;
% Grandchild = john.
Query: Find all pairs of siblings in the database.
?- sibling(A, B).
% Output:
% A = lisa, B = tim ;
% A = tim, B = lisa.
Query: List every child that Bob has.
?- findall(Child, parent(bob, Child), Children).
% Output: Children = [lisa, tim].
```

Ex4

https://swish.swi-prolog.org/p/prolog-family-tree.pl

```
/* Facts */
male(jack).
male(oliver).
male(ali).
male(james).
male(simon).
male(harry).
female(helen).
female(sophie).
female(jess).
female(lily).
 parent_of(jack,jess).
parent_of(jack,lily).
parent_of(helen, jess).
parent_of(helen, lily).
parent_of(oliver,james).
parent_of(sophie, james).
parent_of(jess, simon).
```

```
parent_of(ali, simon).
parent_of(lily, harry).
parent_of(james, harry).

/* Rules */
father_of(X,Y):- male(X),
   parent_of(X,Y).
 mother_of(X,Y):- female(X),
   parent_of(X,Y).
 grandfather_of(X,Y):- male(X),
   parent_of(X,Z),
   parent_of(Z,Y).
 grandmother_of(X,Y):- female(X),
   parent_of(X,Z),
   parent_of(Z,Y).
 sister_of(X,Y):- %(X,Y or Y,X)%
   female(X),
   father_of(F, Y), father_of(F,X),X \= Y.
 sister_of(X,Y):- female(X),
   mother_of(M, Y), mother_of(M,X),X \= Y.
 aunt_of(X,Y):- female(X),
   parent_of(Z,Y), sister_of(Z,X),!.
 brother_of(X,Y):- %(X,Y or Y,X)%
   male(X),
   father_of(F, Y), father_of(F,X),X \= Y.
 brother_of(X,Y):- male(X),
   mother_of(M, Y), mother_of(M,X),X \= Y.
 uncle_of(X,Y):-
   parent_of(Z,Y), brother_of(Z,X).
ancestor_of(X,Y):- parent_of(X,Y).
ancestor_of(X,Y):- parent_of(X,Z),
   ancestor_of(Z,Y).
```

Queries

1. Is Jack the father of Jess?
   father_of(jack, jess).
2. Is Helen the mother of Lily?
   mother_of(helen, lily).
3. Are Jess and Lily sisters?
   sister_of(jess, lily).
4. Is Oliver the grandfather of Harry?
   grandfather_of(oliver, harry).
5. Is Ali an ancestor of Harry?
   ancestor_of(ali, harry).

   ⛃ Single-Variable Queries (Finding One Person)
   These ask Prolog to find a specific person. Press the Semicolon (;) key
   Who is the father of Simon?
   father_of(X, simon).
6. Who is the mother of James?
   mother_of(X, james).
7. Who are the children of Lily?
   parent_of(lily, Child).
8. Who is the grandmother of Harry?
   grandmother_of(X, harry).
9. Who is the uncle of Harry?
   uncle_of(X, harry).

10. Who is the aunt of Simon?
    aunt_of(X, simon).
11. Does Harry have any brothers?
    brother_of(Brother, harry).
    □ Multi-Variable & Database Queries (Finding Pairs/Lists)
    These look for multiple unknown elements or map out the entire database.
13. Find all father-child pairs in the tree:
    father_of(Father, Child).
14. Find all sisters in the family:
    sister_of(Sister, Sibling).
15. Who are all the descendants of Helen? (All people she is an ancestor to)
    ancestor_of(helen, Descendant).
16. Who are all the ancestors of Simon?
    ancestor_of(Ancestor, simon).
17. Find all grandfathers and their respective grandchildren:
    grandfather_of(Grandfather, Grandchild).

    ● Complex/Filtered Queries
    These combine multiple rules together to find specific configurations.
18. Find all males who are also fathers:
    male(X), father_of(X, _).
19. Who has both Jack and Helen as parents? (Finds full siblings)
    parent_of(jack, X), parent_of(helen, X).
20. Find anyone who is a parent but does NOT have any grandchildren in this tree:
    parent_of(X, Y), \+ parent_of(Y, _).

Ex5 Airport_flights.pl

```
/* Facts: Airports (City, Country) */
airport(london, uk).
airport(paris, france).
airport(frankfurt, germany).
airport(dubai, uae).
airport(delhi, india).
airport(singapore, singapore).
/* Facts: Direct Flights (FlightNumber, Origin, Destination, Duration in Hours) */
flight(ba101, london, paris, 1).
flight(lh202, paris, frankfurt, 1.5).
flight(ek303, frankfurt, dubai, 6).
flight(ek404, london, dubai, 7).
flight(ai505, dubai, delhi, 3.5).
flight(sq606, delhi, singapore, 5.5).
/* Rules */
% 1. Check if a direct flight exists between two cities
direct_route(From, To) :-
  flight(_, From, To, _).
% 2. Find a route with exactly one layover (1 stop connection)
one_stop_route(From, To, Layover) :-
  flight(_, From, Layover, _),
  flight(_, Layover, To, _).
% 3. Calculate total flight duration for a 1-stop connection
one_stop_duration(From, To, TotalTime) :-
  flight(_, From, Layover, Time1),
  flight(_, Layover, To, Time2),
  TotalTime is Time1 + Time2.
% 4. Recursive Rule: Find if any path exists between two cities (Handles multi-stops)
```

```
% Base Case: Direct flight
has_route(From, To) :-
  direct_route(From, To).
% Recursive Case: Fly to an intermediate city, then find a route from there
has_route(From, To) :-
  direct_route(From, Via),
  has_route(Via, To).
```

Queries

 Yes/No Verification Queries [1]

1. Is there a direct flight from London to Paris?
   direct_route(london, paris).
2. Can I fly directly from Paris to Delhi?
   direct_route(paris, delhi).
3. Is it possible to travel from London to Singapore eventually (via connections)?
   has_route(london, singapore).

 Single-Variable Queries (Finding Options)

4. Where can I fly to directly from London?
   direct_route(london, Destination).
5. Which cities have a direct flight into Dubai?
   direct_route(Origin, dubai).
6. What is the flight number of the flight leaving Delhi?
   flight(FlightNum, delhi, _, _).
7. If I want to go from London to Dubai with exactly one stop, where will I layover?
   one_stop_route(london, dubai, Layover).
8. Where can I travel to if I start in Frankfurt and make exactly one stop?
   one_stop_route(frankfurt, Destination, _).

 Calculations and Multi-Variable Queries

9. Find all direct flight routes and their durations:
   flight(_, Origin, Destination, Duration).
10. How long will I spend in the air if I fly London -> Paris -> Frankfurt?
    one_stop_duration(london, frankfurt, Hours).
11. Find a 1-stop route from London to Delhi and calculate the total flying time:
    one_stop_route(london, delhi, Layover), one_stop_duration(london, delhi, TotalTime).
    Use code with caution.
12. List all pairs of cities that are connected by a 1-stop flight:
    one_stop_route(CityA, CityB, _).

● Advanced Filtered Queries

13. Find all direct flights that take longer than 5 hours:
    flight(Num, From, To, Duration), Duration > 5.0.
14. Find short direct flights (under 2 hours):
    flight(Num, From, To, Duration), Duration < 2.0.
15. Are there any flights where the destination country is the UK?
    flight(_, _, Destination, _), airport(destination, uk).
16. Find a flight that departs from a city in France:
    airport(City, france), flight(_, City, Destination, _).
17. Find all destinations reachable from London (direct or indirect):
    has_route(london, Destination).
18. Find any flight whose flight number starts with 'ek' (using wildcard sub-string check):
    flight(FlightNum, _, _, _), sub_atom(FlightNum, 0, 2, _, ek).
19. Find a destination you can reach from Paris that requires a total flying time of exactly 7.5 hours (with 1 stop):
    one_stop_duration(paris, Dest, 7.5).
20. Is there any city you can fly to from London that does NOT have a flight to Singapore?
    direct_route(london, NextCity), \+ has_route(NextCity, singapore).

Ex6. Colonel West - The Criminal Problem

This is a famous tracking problem found in Russell & Norvig's chapter on inference in First-Order Logic.

The Narrative:

It is a crime for an American to sell weapons to hostile nations.
The country Nono, an enemy of America, has some missiles.
All of its missiles were sold to it by Colonel West, who is American.

```
% Facts
american(west).
enemy(nono, america).
missile(m1).
owns(nono, m1).

% "All of its missiles were sold to it by Colonel West"
sells(west, X, nono) :- missile(X), owns(nono, X).
% "Missiles are weapons"
weapon(X) :- missile(X).
% "An enemy of America is a hostile nation"
hostile(X) :- enemy(X, america).
% "It is a crime for an American to sell weapons to hostile nations"
criminal(X) :- american(X), weapon(Y), sells(X, Y, Z), hostile(Z).
```

Queries

```
?- criminal(west). % Output: true
```

Verification Queries (Yes/No)

These ask Prolog to confirm whether a specific fact or relationship holds true.

Query1: Is Nono an enemy of America?

```
?- enemy(nono, america).
% Output: true
```

Query2: Did West sell a missile to Nono?

```
?- sells(west, m1, nono).
% Output: true
```

Query 3: Is m1 classified as a weapon?

```
?- weapon(m1).
% Output: true
```

2. Information Retrieval Queries (Finding Who or What)

These use uppercase letters as variables, prompting Prolog to find and list all matching items.

Query4: Who is a criminal?

```
?- criminal(Who).
% Output: Who = west.
```

Query5: What items does Nono own?

```
?- owns(nono, Item).
% Output: Item = m1.
```

Query6: To whom did West sell a weapon?

```
?- sells(west, Weapon, Buyer).
% Output: Weapon = m1, Buyer = nono.
```

Complex & Compound Queries (Using , for AND)

Query7: To explore the full power of First-Order Logic in Prolog, we can look at queries that involve conjunctions (AND), disjunctions (OR), negation, and recursive rules. See advanced query patterns using both the Colonel West and the Family Tree domains. You can string multiple conditions together in a single query line. Prolog will only return true or bind variables if all conditions are satisfied simultaneously.

Query8: Find an American who sold a weapon to an enemy nation.

```
?- american(X), weapon(Y), sells(X, Y, Z), enemy(Z, america).
```

% Output: X = west, Y = m1, Z = nono.
Query9: Is there any missile owned by an enemy that West did not sell? (Checking for other suspects).
?- missile(M), owns(nono, M), \+ sells(west, M, nono).
% Output: false. (West sold all of Nono's missiles).

Aggregation Queries (Finding All Results)
In First-Order Logic, you often want to find the entire set of objects that satisfy a predicate. Prolog uses *findall* for this.
Query10: Collect all hostile nations into a single list.
?- findall(Nation, hostile(Nation), List).
% Output: List = [nono].

Trace a Multi-Step Inference
If you want to see exactly how Prolog navigates the First-Order Logic rules step-by-step to conclude that West is a criminal, you can activate the tracer before running the query:
?- trace.
?- criminal(west).
What the trace output reveals:

1. Call: criminal(west) — Prolog attempts to prove West is a criminal.
2. Call: american(west) — Matches a direct fact. (Success)
3. Call: weapon(Y) — Looks at rules; finds weapon(X) :- missile(X).
4. Call: missile(Y) — Matches missile(m1). (Success, binds Y = m1)
5. Call: sells(west, m1, Z) — Evaluates rule sells(west, X, nono) :- missile(X), owns(nono, X). (Success, binds Z = nono)
6. Call: hostile(nono) — Evaluates rule hostile(X) :- enemy(X, america). (Success)
7. Exit: criminal(west) — All conditions met. Returns true.

Core Limitations: Prolog vs. Full FOL

Prolog trades the limitless expressiveness of Full First-Order Logic (FOL) for computational efficiency and practical execution. While Full FOL allows any combination of logical operators and quantifiers, Prolog operates on a strict subset called **Horn clause logic**.This architectural difference shapes Prolog's distinct limitations:

The Expressiveness Gap

No Disjunctions: Full FOL allows for statements like $A \vee B$ (A or B). Prolog enforces that clauses must have at most one positive literal (i.e., $A \leftarrow B \wedge C$). You cannot natively express "either X is true or Y is true" without writing separate rules for alternate cases. Wikipedia +2

No Universal Existentials: Full FOL allows mixed quantifiers, such as stating every person has a brain ($\forall x \exists y$ Has(x, y)). Prolog prohibits existential quantification in the conclusion of a rule, limiting what can be queried directly. YouTube · Stanford Online +2

==========